the interceptor pipeline is driven by a `Promise.resolve`-scheduled microtask chain, and the final network call runs in a later macrotask. The synchronous stack at the sink contains only the network wrapper and a few framework adapters; the debugger's asynchronous stack trace identifies which microtask scheduled the network call but not which function constructed the value in `body.order_sig_v2`. Recovering the anchor requires stitching synchronous stacks, asynchronous call provenance, and cross-turn value flows into one structure rather than reading any single frame.

d) Why an LLM agent without static–dynamic coordination falls short: An agent that begins from keyword search wades through decoy digest paths surfaced by structural uncertainty and, latching onto the most prominent digest, recovers only a generic SHA-256 utility whose body cannot explain why `body.order_sig_v2` in particular is produced. An agent that follows traces uniformly must inspect thousands of framework callbacks before reaching the interceptor. An agent that starts from the network sink sees the body only after the signature is inserted and finds no target-specific function on the synchronous stack. These failures arise not from weak reasoning but because no single source of evidence suffices and the agent has no mechanism to make the three constrain each other.

APPENDIX C
PROBLEM DEFINITION: WORKED CASES

This appendix expands §III; the deletion-based operational test for target-specificity is stated in the body (§III).

a) Three nested cases: Case 1 (anchor sandwiched between generic dispatch and a generic utility): in a chain $A \to B \to C \to D$ where A is a click handler, B a generic interceptor-pipeline dispatcher, C a checkout-specific sealing interceptor that assigns `body.order_sig_v2`, and D a reusable SHA-256 routine, the anchor is C: A and B route any request, while D is a generic hash whose code cannot by itself explain why this particular field is produced. *Case 2 (anchor above the literal binding site):* if A is a generic submit handler that writes whatever its callee returns, B is dedicated to computing `search_sig`, and C is the underlying hash, the anchor is B even though the literal assignment occurs in A, because B is the highest chain function whose body is dedicated to the target. *Case 3 (anchor embedded in a mixed-purpose function):* when the target value is produced by an inline statement such as `ctx.body[targetField] = digest(...)` inside a function whose remaining body handles unrelated request types, the enclosing function is still target-specific and the anchor rule selects it.

b) Existence of f^* and the no-anchor case: The anchor rule presupposes that at least one target-specific function exists on the chain. Behaviors realized entirely by composition of generic utilities (for example, a request body produced by a stock serializer with no target-specific glue) admit no anchor under the rule. Our benchmark protocol excludes such tasks during construction: a candidate behavior is admitted only if expert annotators agree that at least one chain function passes the deletion-based test of §III. Methods evaluated on this benchmark may therefore assume the existence of f^* and are not required to abstain. Extending the framework to abstain on no-anchor behaviors is left to future work.

APPENDIX D
APPROACH DETAILS

This appendix expands the condensed cost model of information-gain breakpoint selection (§IV-C) and the value-match predicate of async causal stitching (§IV-D).

a) Cost model $c(b_i)$: We instantiate $c(b_i)$ as the expected number of times b_i fires within a single task trigger, capped at a per-category constant $c_{\max}$. Breakpoints inside one-shot event handlers receive $c(b_i) = 1$. Breakpoints inside Promise continuations or microtask callbacks receive $c(b_i)$ equal to the static fan-in of the enclosing dispatch site, also capped at $c_{\max}$. Breakpoints inside rendering, polling, or interval callbacks are clipped at $c_{\max}$ a priori, since their hit count is governed by viewport and timer state rather than by the task trigger. The numerical value of $c_{\max}$ is fixed per category; the loop's behavior is insensitive to its exact value provided that recurrent callbacks are penalized strictly more than one-shot handlers.

b) Value match for data edges: For *long strings and byte arrays* (length $\geq \ell_{\text{long}}$), match is exact equality on canonical bytes, hashed and indexed to allow $O(1)$ pairwise lookup. For *structured objects*, match requires identical key schemas and a key-sorted shallow value hash. For *short primitives* such as booleans, small integers, and short strings, match is equality discounted by the global occurrence frequency of the value across all collected hits, suppressing edges induced by ubiquitous values such as `true` or empty strings. For *containment relationships*, where one value is a substring or subarray of another, a partial data edge is added with a lower weight $w_{\text{partial}} \in (0, 1)$, capturing the common case of intermediate digests being concatenated into a final signature. This criterion is conservative enough that random framework values do not produce data edges, yet permissive enough that the closure-held session pepper observed at one hit is linked to the digest material observed at a later hit in a different microtask.

c) Per-stage computational cost: The bound of §IV-E decomposes as follows. The breakpoint proposer issues one LLM call per iteration to enumerate candidate breakpoints with predicted outcomes. The subsequent Bayesian update evaluates the LLM likelihood $P(o_{i,j} \mid f, b_i)$ as a binary consistency judgment over the top-k slice of H_{t-1}, at most $k \cdot M$ batched judgments per iteration, batched into one LLM call per breakpoint. Causal stitching incurs one LLM classifier call per newly visited node in the backward BFS, bounded by the causal-graph depth along reachable paths from the sink. CDP traffic is dominated by the at most $k \leq 3$ breakpoints set per iteration plus a constant per-hit cost for capturing the 4-tuple (S_h, A_h, V_h, T_h).

d) Pseudocode: Algorithms 2 and 3 restate the breakpoint selector and the graph-stitching and anchor-recovery

Algorithm 2 INFOGAINPROBE: information-gain breakpoint selection

Require: hypothesis H_t, causal graph G_t
Ensure: selected breakpoint b^*

$B_{\text{cand}}, \{(o_{i,j}, q_{i,j})\} \leftarrow \text{LLMPROPOSE}(H_t, G_t)$
for all $b_i \in B_{\text{cand}}$ **do**
  $\text{IG}_i \leftarrow \mathcal{H}(H_t)$
  for all outcome $o_{i,j}$ of b_i **do**
    $H' \leftarrow \text{BAYESUPDATE}(H_t, b_i = o_{i,j})$
    $\text{IG}_i \leftarrow \text{IG}_i - q_{i,j} \cdot \mathcal{H}(H')$
  end for
  $\text{IG}_i \leftarrow \text{IG}_i / (c(b_i) + \epsilon)$
end for

return $b^* \leftarrow \arg\max_{b_i} \text{IG}_i$

Algorithm 3 STITCHCAUSALGRAPH and Anchor Recovery

Require: previous graph G_{t-1}; new hits $\{h_1, \ldots, h_n\}$; sink node n_{sink}
Ensure: updated graph G_t; anchor candidate set A_t

$G_t \leftarrow G_{t-1}$
for all hit h in $\{h_1, \ldots, h_n\}$ **do**
  $(S_h, A_h, V_h, T_h) \leftarrow \text{CAPTURE}(h)$
  add nodes and sync edges from S_h to G_t
  add async edges from A_h to G_t
  for all prior hit h' with $T_{h'} \neq T_h$ **do**
    for all $v_i \in V_h, v_j \in V_{h'}$ **do**
      if $\text{VALUEMATCH}(v_i, v_j)$ **then**
        add data edge from h to h' to G_t
      end if
    end for
  end for
end for
$A_t \leftarrow \emptyset$; $Q \leftarrow \{n_{\text{sink}}\}$; VISITED $\leftarrow \{n_{\text{sink}}\}$
while $Q \neq \emptyset$ **do**
  $n \leftarrow \text{DEQUEUE}(Q)$
  if $\text{ISTARGETSPECIFIC}(n.f, d)$ **and** $\text{REACHESTRIGGER}(n, G_t)$ **then**
    $A_t \leftarrow A_t \cup \{n\}$ {do not expand predecessors of n}
  else
    for all predecessor n' of n in G_t **do**
      if $n' \notin$ VISITED **then**
        enqueue n' into Q; VISITED $\leftarrow$ VISITED$\cup\{n'\}$
      end if
    end for
  end if
end while
return G_t, A_t

procedure of §IV-C and §IV-D (the overall co-refinement loop is Algorithm 1 in the body); they are provided for implementers and are not required to follow the main text.

TABLE IV
IMPLEMENTATION ENVIRONMENT.

Component	Version
Node.js	20.19.0
Chrome DevTools Protocol	Chrome 138.x
Acorn	8.16.0
Astring	1.9.0
chrome-remote-interface	0.33.3
ws	8.21.0

APPENDIX E
HYPERPARAMETERS AND CLOSED FORMS

This appendix gives the implementation environment and the closed forms deferred from §IV-F. All values are implementation defaults; no grid search or held-out calibration is performed, and every value is fixed across datasets.

a) LLM configuration: The system uses an OpenAI-compatible endpoint with a unified decoding configuration: model `DeepSeek-V4-Pro` [51], temperature 0, JSON-mode responses (auto fallback on failure), and a retry budget of 3 for JSON decoding.

b) Likelihood factorization: For each candidate function $f \in F_C$, the system defines a multiplicative likelihood

$$L_t(f) = \text{clip}\Big(L_{\text{val}}(f) \cdot L_{\text{anchor}}(f) \cdot L_{\text{pred}}(f),\ L_{\min}, L_{\max}\Big), \tag{1}$$

with global clipping bounds $L_{\min}, L_{\max}$. Let f_{hit} denote the function associated with the observed breakpoint:

$$L_{\text{val}}(f) = \begin{cases} L_{\text{val}}^{+}, & f = f_{\text{hit}} \wedge \text{valueMatch} \\ L_{\text{val}}^{-}, & f = f_{\text{hit}} \wedge \neg\text{valueMatch} \\ 1, & \text{otherwise.} \end{cases} \tag{2}$$

Let f^* be the anchor candidate and $s(f) \in [0,1]$ its LLM score:

$$L_{\text{anchor}}(f) = \begin{cases} b_{\text{anchor}} + s(f^*), & f = f^* \\ L_{\text{reject}}, & f \neq f^*,\ s(f) < \theta_{\text{anchor}} \\ 0.5, & f \neq f^*,\ s(f) \geq \theta_{\text{anchor}} \\ 1, & \text{otherwise.} \end{cases} \tag{3}$$

$L_{\text{pred}}(f)$ is produced by the breakpoint-selection module; if unavailable, $L_{\text{pred}}(f) = 1$.

c) Bayesian update:

$$\tilde{H}_{t+1}(f) = H_t(f) \cdot L_t(f), \qquad H_{t+1}(f) = \frac{\tilde{H}_{t+1}(f)}{\sum_{g \in F_C} \tilde{H}_{t+1}(g)}. \tag{4}$$

If the denominator is zero, a uniform fallback distribution is used.

d) Redundancy and convergence: With $\hat{f} = \arg\max_f H_t(f)$,

$$\text{Redundant}(\hat{f}) = \text{LLMValid}(\hat{f}) \wedge \big(\text{P0Aux}(\hat{f}) \vee \text{CausalAux}(\hat{f})\big), \tag{5}$$

$$\text{Converged} = \big(H_t(\hat{f}) \geq \theta_{\text{conf}}\big) \wedge \text{Redundant}(\hat{f}). \tag{6}$$

TABLE V
DEFAULT HYPERPARAMETERS, FIXED ACROSS DATASETS.

Symbol	Value	Description
temperature	0	LLM decoding temperature
K	3	Breakpoint-selection focus size
$T_{\max}$	20	Maximum iterations
$L_{\min}$	0.1	Likelihood lower bound
$L_{\max}$	10	Likelihood upper bound
L_{val}^{+}	2.5	Value-match reward
L_{val}^{-}	0.4	Value-mismatch penalty
b_{anchor}	8.0	Anchor base boost
L_{reject}	0.1	Anchor rejection penalty
θ_{conf}	0.9	Convergence confidence threshold
θ_{anchor}	0.7	Anchor acceptance threshold
ε	10^{-6}	IG numerical stability
τ	1.0	Structural prior temperature
w_{ast}	1	AST weight
w_{api}	0.8	API weight
w_{ent}	1	Entropy weight
w_{sink}	0.5	Sink proximity weight
D	7	Graph traversal depth
q_{p_0}	0.7	Structural prior percentile cutoff

e) Structural prior: The structural score combines four components,

$$S(f) = w_{\text{ast}}S_{\text{ast}}(f)+w_{\text{api}}S_{\text{api}}(f)+w_{\text{ent}}S_{\text{ent}}(f)+w_{\text{sink}}S_{\text{sink}}(f), \tag{7}$$

where S_{ast} encodes AST template similarity, S_{api} API-signature relevance, S_{ent} entropy-related behavior density, and S_{sink} static proximity to sink nodes. The prior is a temperature-scaled softmax,

$$p_0(f) = \frac{\exp\big((S(f) - \max_g S(g))/\tau\big)}{\sum_{g\in F_C}\exp\big((S(g) - \max_h S(h))/\tau\big)}, \tag{8}$$

and $H_0(f) = p_0(f)$. The sink-proximity term uses the static call graph G and sink set $\mathcal{S}$: $S_{\text{sink}}(f) = 1/(d_G(f,\mathcal{S}) + 1)$, and 0 if unreachable.

f) Breakpoint-selection acquisition objective: $IG(b) = H(H_t) - \mathbb{E}_o[H(H_t \mid o)]$ and $\text{score}(b) = IG(b)/(c(b) + \varepsilon)$.

APPENDIX F
BENCHMARK CATEGORY EXAMPLES

Table VI lists, for each of the five ANCHORBENCH categories defined in §V-C, the target observable and a representative maintainer-verified anchor named by its captured-bundle identifier.

TABLE VI
THE FIVE BEHAVIOR CATEGORIES OF ANCHORBENCH, WITH A REPRESENTATIVE ANCHOR NAMED BY ITS CAPTURED-BUNDLE IDENTIFIER.

Category	Target observable	Representative anchor (role)
Request signing & token derivation	proof/seal token from fields + runtime context (e.g. `ap_`-prefixed `account_proof`)	`encodeAccountProofEnvelope`: folds tuple/config/runtime state into the token, above the hash it calls
State encoding	compact stable encoding of form + UI/route state (e.g. `state_code`)	`encodeStateEnvelope`: selects the reducer and formats the code
Byte-array transformation	byte payload materialized then serialized (e.g. `Uint8Array`→`base64url` `byte_payload`)	`encodeByteArrayEnvelope`: materializes and encodes the payload
Browser fingerprinting	compact digest of navigator/screen/timezone/media/canvas (e.g. 12-char `browser_fingerprint`)	`assembleBrowserFingerprint`: selects surfaces and formats the digest
Request transformation	deterministic value written into an outgoing field (e.g. `X-Transform-Key`)	`buildRequestTransform`: packages the value before the network wrapper

APPENDIX G
BENCHMARK CONSTRUCTION DETAILS

This appendix expands the condensed corpus, oracle, and annotation description of §V-C. The body is self-contained without it; these details support reproduction.

a) Corpus and packaging: Each case is bundled with `rollup` and hardened with `javascript-obfuscator` under a fixed seed (local-identifier mangling, string-array extraction, control-flow flattening; source maps off). We deliberately keep module-scope function names (`renameGlobals` off), modeling builds that harden locals, strings, and control flow but retain top-level names, so we can test whether the structural prior helps even when lexical baselines still have a name signal; identifier erasure is studied separately at obfuscation-ladder tier T1 (§V-D3). Each case splits into `agent_visible/` (the `task.json` and a DevTools capture of exactly the main-context scripts a debugger exposes at the trigger, i.e. the corpus C) and a never-visible `agent_hidden/` (un-obfuscated `src/`, build/grading scripts, oracle).

b) Oracle and grading: Grading is strictly top-1: a returned complete function is matched to a role entry by body SHA-256 (and a whitespace-normalized variant) with a $\{\text{file}, \text{offset}\}$/span-containment fallback; S_d is the matched score, and an exact anchor match counts toward strict accuracy. Because the oracle defines the target, it is established *entirely by human annotation*: annotators may consult an instrumented trace as evidence, but all chain, role, and anchor judgments are human, with the deletion-based criterion of Section III used as a written guideline rather than a decidable procedure. Each role assignment is recorded with a written rationale in the released oracle, and behaviors for which no chain function satisfies the deletion-based criterion are excluded during construction. A build-time verifier re-triggers each case to confirm the observable reproduces and re-resolves every